Motorcycle Service Web App
Analysis and Design Document : Assignment 1

Student: Fechete Serban Tudor
Group: 30431

Table of Contents

Contents

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to provide a management system for a motorcycle service center or garage.

1.2 Functional Requirements

The application has the following functional requirements:

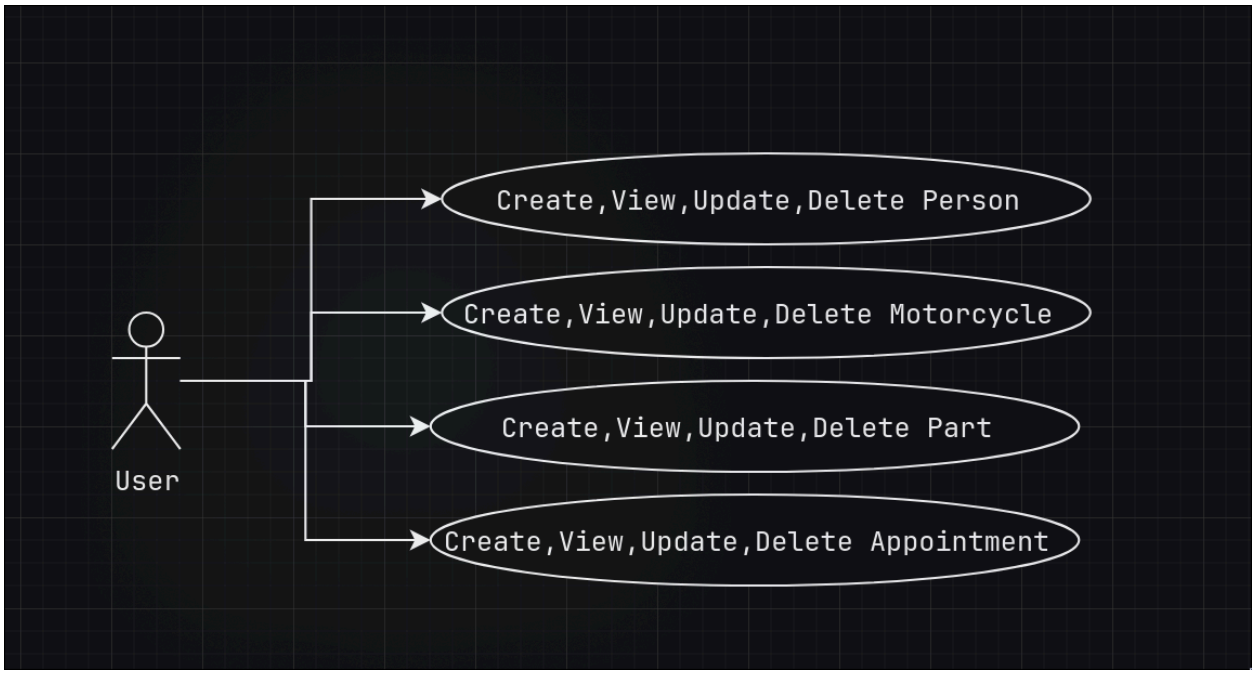
- Allow the creation and management of customer profiles (Persons), capturing details such as their name, age, password, and a unique email address.
- Allow users to register and manage multiple motorcycles associated with a specific owner, tracking the brand, model, manufacture year, and license plate.
- Allow the scheduling and tracking of service appointments for specific motorcycles.
- Allow the application to associate and track specific parts used during a given service appointment.

1.3 Non-functional Requirements

The application has the following non-functional requirements:

- Use Angular to implement the frontend client interface.
- Use Java and the Spring Boot framework to build the backend REST API.
- Use PostgreSQL as the underlying relational database to store all application data.
- Use an ORM to handle database interactions, automatically update the database schema, and manage entity relationships.
- Implement database-level validation to ensure data integrity, such as enforcing non-nullable fields and unique email constraints.
- Use a layered architecture for better organization and separation of concerns.

2. Use-Case Model

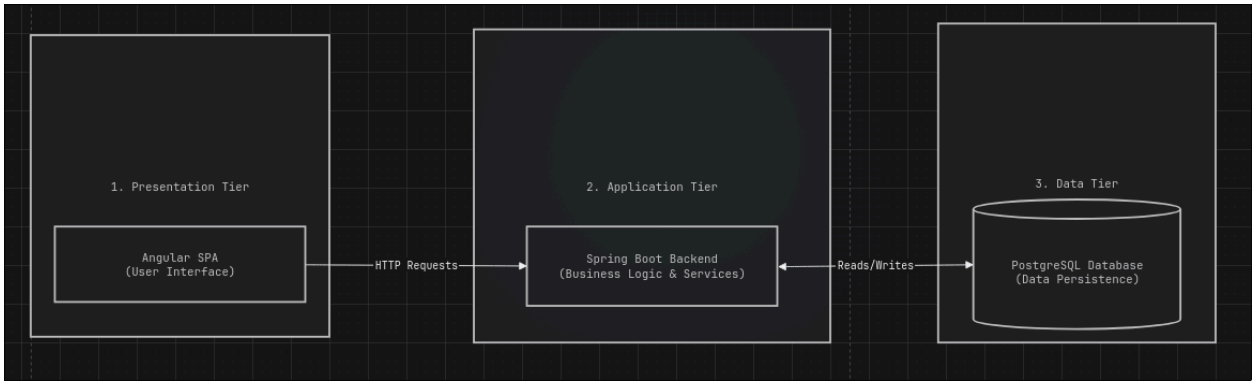


Description of the “Fetch all motorcycles” use-case:

- Use case goal: Fetching all registered motorcycles from the system via the /motorcycle endpoint.
- Primary actor: The user of the application.
- Main success scenario: Database connection succeeds, data is fetched correctly by the backend service, and a complete list of Motorcycle entities is returned to the client.
- Extensions:
    - Alternate scenario of success: Database connection succeeds, but no motorcycles are contained in the response because none have been registered yet.
    - Failure: Either the database connection fails, the request to /motorcycle times out, or the application encounters an internal error. The user will get a corresponding error message in the frontend.

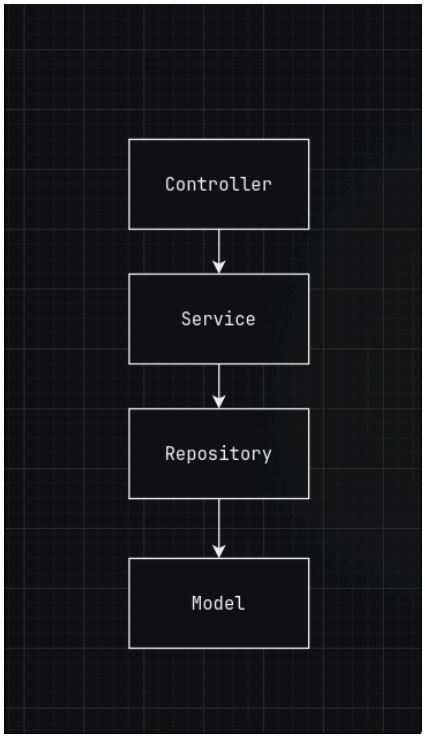
3. System Architectural Design

3.1 Architectural Pattern Description



- The presentation layer handles the user interface and client-side interactions, as well as the API endpoints that expose backend functionality.
- The logic layer acts as the intermediary between the presentation and data layers, containing the core business rules and processing logic.
- The data layer is responsible for securely storing, retrieving, and managing the application's persistent data.

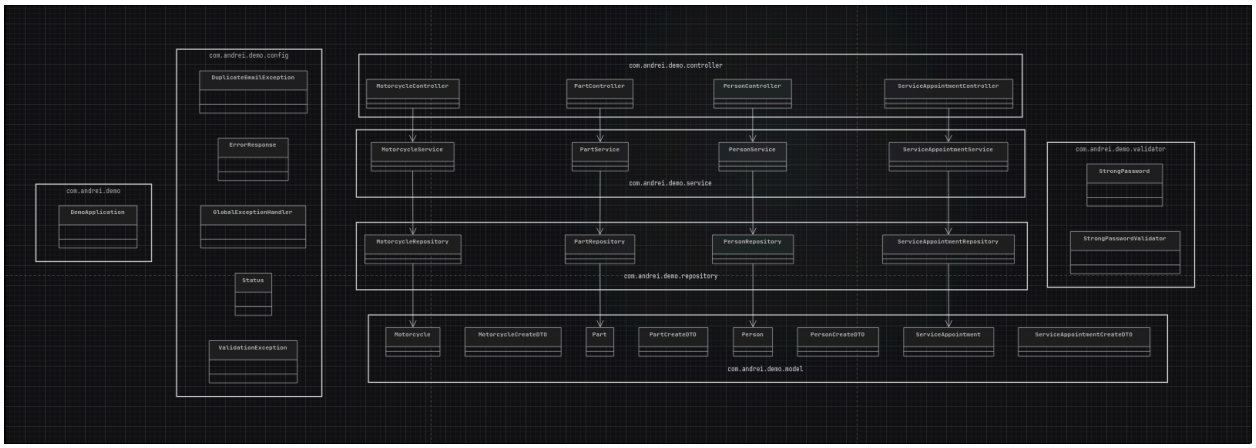
3.2 Diagrams



- Controllers : The entry point. They receive HTTP requests, trigger DTO validation, and route tasks to the Service layer without holding any business logic.
- Services: They enforce business rules, convert frontend DTOs into database Entities, and delegate data operations to the Repository.
- Repositories: Manage all database interactions. Using Spring Data JPA, they automatically translate method calls into PostgreSQL queries to handle Entities.
- Models: The objects moving through these layers, taking two forms: DTOs and Entities.

4. Class Design

4.1 Package + Class Diagram



5. Data Model